REXX TCP Socket Interface (C) - Documentation

A lightweight TCP client/server socket API designed for use with `CREXX`, providing essential networking operations accessible from REXX or compatible environments.

Table of Contents

- Overview
- Data Structure
- API Procedures
  - socketcreate
  - socketconnect
  - socketsend
  - socketsendall
  - socketrecv
  - socketrecvline
  - socketclose
  - socketsetblocking
  - socketpendingbytes
  - socketisconnected
  - socketpeerinfo
  - socketlocalinfo
  - socketsettimeout
  - socketshutdown
  - socketnodelay
  - socketkeepalive
  - socketbind
  - socketlisten
  - socketaccept
  - socketlasterror
- Error Codes
- Usage Example

Overview

This module provides a cross-platform TCP socket interface for CREXX. It allows scripts or applications to create client and server sockets, connect, send and receive data, and handle network events using simple procedural calls.

Data Structure

```
typedef struct {
#ifdef _WIN32
    SOCKET sock;
#else
    int sock;
#endif
    int status;
    int is_server;
    int default_timeout;
    int last_error;
    char last_error_msg[128];
    char *linebuf;
    int linebuf_used;
    int linebuf_size;
} TcpSocket;
```

API Procedures (Detailed)

1. `socketcreate`

- **Description:**
  Allocates and initializes a new TCP socket structure for later use as a client or server. This is always your first step before any other socket operation!

- **Typical Use:**
  Call this before you connect, bind, or listen. It gives you a handle (think: socket “token”) you use for everything else.
- **Returns:**
  - On success: a unique socket handle (an integer value)
  - On failure: `-8` (out of memory, very rare)
- **Good to know:**
  The returned handle must be passed to all other functions. Always call `socketclose` to free resources when done. If you forget, your program might leak memory or sockets.
- **Sample:**

```
sock = socketcreate()
if sock < 0 then say "Could not create socket!"
```

---

2. `socketconnect`

- **Description:**
  Connects your socket to a remote server using a hostname (or IP) and port number. This is your entry into the world—make a connection, then send/receive data.
- **Typical Use:**
  For clients: connect to HTTP, Telnet, or any TCP service.
- **Parameters:**
  - `sock` (int): The handle from `socketcreate`
  - `host` (string): Target hostname (e.g., "example.com") or IP (e.g., "192.168.1.5")
  - `port` (int): TCP port (e.g., 80 for HTTP)
- **Returns:**
  - `0` on success
  - `<0` on error (see error codes for meaning)
- **Tips:**
  - Always check the return code! If not `0`, call `socketlasterror` for the human-readable error.
  - DNS issues show up as `-4` (hostname resolution failed).
  - If connecting to a local server for testing, use "127.0.0.1".
- **Caveat:**
  This function is synchronous—if the server is down, it may block for a while (until timeout or failure).
- **Sample:**

```
rc = socketconnect(sock, "chatserver.example.org", 4242)
if rc \= 0 then say "Connection failed:" socketlasterror(sock)
```

---

3. `socketsend`

- **Description:**
  Sends a string (or byte buffer) to the connected socket. Use for sending messages, commands, or files.
- **Typical Use:**
  After connecting, send your request or data.
- **Parameters:**
  - `sock` (int): Your socket handle
  - `data` (string): Data to send (may include binary!)
- **Returns:**
  - Number of bytes sent (could be less than requested—rare, but check!)
  - `<0` on error
- **Tips:**
  - TCP may split or buffer data; for all-or-nothing delivery, use `socketsendall`.
  - Data sent is not guaranteed to arrive instantly at the remote end.
- **Caveat:**
  If the socket isn't connected, returns `-6` ("Socket not connected").
- **Sample:**

```
sent = socketsend(sock, "Hello, world!")
if sent < 0 then say socketlasterror(sock)
```

---

4. `socketsendall`

- **Description:**
  Sends all bytes from your string/data, looping as needed until every byte is written or an error occurs. Useful for larger payloads or ensuring complete delivery.
- **Typical Use:**
  For big messages, file transfers, or when you need to be absolutely sure everything was sent.
- **Parameters:**
  - `sock` (int): Socket handle
  - `data` (string): What you want to send
- **Returns:**
  - Number of bytes sent (should equal `length(data)` on success)
  - `<0` on error

- **Tips:**
  - Always check the return value! If it's less than the data length, something went wrong.
  - Useful for protocols where incomplete data is a bug (e.g., binary protocols).
- **Caveat:**
  This can block if the network is slow.
- **Sample:**

```
rc = socketsendall(sock, filedata)
if rc < length(filedata) then say "Send failed!"
```

---

5. `socketrecv`

- **Description:**
  Reads up to a given number of bytes from the socket (but possibly less, depending on what the remote side has sent). Good for chunked reads or streaming data.
- **Typical Use:**
  To get a response after sending a request, or to read incoming data in a loop.
- **Parameters:**
  - `sock` (int): The socket handle
  - `size` (int): Max bytes to read (up to 4096 per call recommended)
- **Returns:**
  - Received data as a string (could be shorter than requested, or empty if connection closed)
- **Tips:**
  - Always check for empty string—could mean error *or* that the remote side closed the connection.
  - For text protocols, you might want to use `socketrecvline` instead.
- **Caveat:**
  May block if no data available, unless the socket is in non-blocking mode or a timeout is set.
- **Sample:**

```
buf = socketrecv(sock, 1024)
if buf = "" then say "Remote closed connection or error."
```

---

6. `socketrecvline`

- **Description:**
  Reads one line (terminated by) from the socket, automatically buffering partial data as needed. Perfect for text protocols.
- **Typical Use:**
  Reading commands, chat messages, or HTTP headers line by line.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - A single line of text (no trailing CR/LF), or empty string if error/disconnect.
- **Tips:**
  - Handles buffering for you—no need to manage partial lines.
  - Useful for REPLs, SMTP, POP3, IRC, etc.
- **Caveat:**
  If a line is longer than 4 MB, you'll get a buffer overflow error (`-10`).
- **Sample:**

```
line = socketrecvline(sock)
if line \= "" then say "Received line:" line
```

---

7. `socketclose`

- **Description:**
  Closes the socket and frees all resources associated with it. Always call this when done to avoid leaks!
- **Typical Use:**
  End of connection, error cleanup, or before exiting your program.
- **Parameters:**
  - `sock` (int): The socket handle
- **Returns:**
  - `0` on success
- **Tips:**
  - Even if a previous operation failed, always close your sockets!
  - Safe to call on an already-closed socket (will do nothing).
- **Caveat:**
  After closing, don't use the handle again.
- **Sample:**

```
call socketclose, sock
```

---

8. `socketsetblocking`

- **Description:**
  Switches the socket between blocking (default) and non-blocking modes.
- **Typical Use:**
  Use non-blocking for GUIs, event loops, or when you want to avoid your program “hanging” waiting for network data.
- **Parameters:**
  - `sock` (int): Socket handle
  - `blocking` (int): `1` for blocking, `0` for non-blocking
- **Returns:**
  - `0` on success, `-1` on error
- **Tips:**
  - Non-blocking mode is advanced: you must handle partial reads/writes, and poll for readiness.
- **Caveat:**
  Some older platforms may not support this cleanly; always test!
- **Sample:**

```
call socketsetblocking, sock, 0   /* non-blocking mode */
```

---

9. `socketpendingbytes`

- **Description:**
  Checks how many bytes are available to read immediately (without blocking).
- **Typical Use:**
  Polling for incoming data, or before reading in non-blocking mode.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - Number of bytes available for reading.
- **Tips:**
  - Useful for “peek and read” logic.
- **Caveat:**
  Will return `0` if nothing to read—doesn’t mean connection is closed.
- **Sample:**

```
if socketpendingbytes(sock) > 0 then data = socketrecv(sock, 4096)
```

---

10. `socketisconnected`

- **Description:**
  Checks if the socket is still connected to the remote host.
- **Typical Use:**
  Before sending/receiving, to make sure the connection is still alive.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - `1` if connected, `0` if not
- **Tips:**
  - “Connected” only means the OS thinks the socket is open; network errors may still occur later.
- **Caveat:**
  Don’t rely solely on this for network health; try sending/receiving too.
- **Sample:**

```
if socketisconnected(sock) then say "Still connected!"
```

---

11. `socketpeerinfo`

- **Description:**
  Gets the remote IP and port for the connected peer as a string.
- **Typical Use:**
  For logging, debugging, or displaying connection info to the user.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - String "IP:port" or empty if not available
- **Tips:**
  - Useful in server code to show who connected.
- **Caveat:**
  After a disconnect, this will be empty.
- **Sample:**

```
say "Remote address:" socketpeerinfo(sock)
```

12. `socketlocalinfo`

- **Description:**
  Gets the local IP and port used by the socket.
- **Typical Use:**
  For servers to confirm which port/IP they are bound to, or for clients using dynamic ports.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - String "IP:port" or empty
- **Tips:**
  - For servers, shows the listening address; for clients, shows the local endpoint.
- **Caveat:**
  On some systems, may show "0.0.0.0:0" before connect/bind.
- **Sample:**

```
say "Local address:" socketlocalinfo(sock)
```

---

13. `socketsettimeout`

- **Description:**
  Sets the default timeout for socket receive operations (in milliseconds).
- **Typical Use:**
  To prevent `socketrecv` or `socketrecvline` from blocking forever.
- **Parameters:**
  - `sock` (int): Socket handle
  - `timeout` (int): Timeout in milliseconds (`0` means no timeout—wait forever)
- **Returns:**
  - `0` on success
- **Tips:**
  - 1000 ms (1 second) is a common timeout for interactive clients.
- **Caveat:**
  Some systems round the timeout; test if your OS supports sub-second resolution.
- **Sample:**

```
call socketsettimeout, sock, 2000  /* 2 seconds */
```

---

14. `socketshutdown`

- **Description:**
  Shuts down part of the socket: receive, send, or both (per standard TCP conventions). Handy for signaling “I’m done sending!” in some protocols.
- **Typical Use:**
  After sending a request and before reading a large response, or for half-close in advanced use.
- **Parameters:**
  - `sock` (int): Socket handle
  - `how` (int): `0` =disable receive, `1` =disable send, `2` =both
- **Returns:**
  - `0` on success, `-20` on error
- **Tips:**
  - Rarely needed for most basic client-server use.
- **Caveat:**
  Once fully shutdown, you must still call `socketclose` to free the handle.
- **Sample:**

```
call socketshutdown, sock, 1   /* done sending */
```

---

15. `socketnodelay`

- **Description:**
  Enables/disables TCP_NODELAY (Nagle’s algorithm), which affects how data is buffered before sending.
- **Typical Use:**
  Enable for real-time protocols (chat, games) where latency matters; disable for bulk transfer.
- **Parameters:**
  - `sock` (int): Socket handle
  - `enable` (int): `1` =on, `0` =off
- **Returns:**
  - `0` on success, `-21` on error
- **Tips:**
  - Most protocols don’t need to change this.
- **Sample:**

```
call socketnodelay, sock, 1   /* low-latency mode */
```

---

16. `socketkeepalive`

- **Description:**
  Enables/disables TCP keepalive, asking the OS to periodically check if the remote is still there.
- **Typical Use:**
  For long-lived connections where you want the OS to detect broken pipes.
- **Parameters:**
  - `sock` (int): Socket handle
  - `enable` (int): `1` =on, `0` =off
- **Returns:**
  - `0` on success, `-22` on error
- **Tips:**
  - OS may wait minutes before sending keepalives; this is not a heartbeat!
- **Caveat:**
  May increase network traffic slightly.
- **Sample:**

```
call socketkeepalive, sock, 1
```

---

17. `socketbind`

- **Description:**
  Binds a socket to a local IP and port, typically used for servers before listening for connections.
- **Typical Use:**
  For server sockets before calling `socketlisten`.
- **Parameters:**
  - `sock` (int): Socket handle
  - `ip` (string): IP to bind to ("" or "0.0.0.0" for all interfaces)
  - `port` (int): Port to listen on (1024+ for user, 80/443 for web servers—may need admin!)
- **Returns:**
  - `0` on success, `-30` on error
- **Tips:**
  - If you get “address already in use”, another server is using this port.
- **Caveat:**
  Bind before listen, and only once per socket.
- **Sample:**

```
call socketbind, sock, "0.0.0.0", 8080
```

---

18. `socketlisten`

- **Description:**
  Puts a bound socket into listening mode, accepting incoming connections. Required for server sockets!
- **Typical Use:**
  Call after `socketbind` to start accepting clients.
- **Parameters:**
  - `sock` (int): Socket handle
  - `backlog` (int): Max pending connections (use 5-10 for most servers)
- **Returns:**
  - `0` on success, `-31` on error
- **Tips:**
  - Larger backlog = more pending clients before accept is called.
- **Caveat:**
  If you forget to call this, `socketaccept` won’t work!
- **Sample:**

```
call socketlisten, sock, 5
```

---

19. `socketaccept`

- **Description:**
  Accepts a new incoming connection on a listening server socket. Returns a new socket handle for communicating with the client.
- **Typical Use:**
  Servers: call in a loop to accept new clients.
- **Parameters:**
  - `sock` (int): Listening socket handle

- **Returns:**
  - New socket handle on success, `<0` on error
- **Tips:**
  - Call `socketpeerinfo` on the new handle to see who connected!
- **Caveat:**
  This function blocks until a client connects (unless in non-blocking mode).
- **Sample:**

```
client = socketaccept(sock)
if client > 0 then say "Client connected!"
```

---

20. `socketlasterror`

- **Description:**
  Returns the last error code and a human-readable message for the given socket.
- **Typical Use:**
  After any function fails, call this to get more info.
- **Parameters:**
  - `sock` (int): Socket handle
- **Returns:**
  - String in the form "code message" (e.g., "-4 Hostname resolution failed")
- **Tips:**
  - Always print this when debugging network issues!
- **Sample:**

```
say "Last error:" socketlasterror(sock)
```

Error Codes

Code	Meaning
-1	Cannot create socket
-2	Connect failed
-3	Send failed
-4	Hostname resolution failed
-5	Receive/read failed
-6	Socket not connected
-7	Timeout
-8	Memory allocation failed
-10	Line buffer overflow
-20	Shutdown failed
-21	TCP_NODELAY setsockopt failed
-22	KEEPALIVE setsockopt failed
-30	Bind failed
-31	Listen failed
-32	Accept failed
0	Success

Important Note: Server Demonstrator

Server-side functionality is provided as a demonstrator only.

> **Warning:** This socket interface does not support multi-threading or asynchronous operations. In a real-world server, you must handle each client connection in a separate thread or process to avoid blocking the entire server. This implementation is intended for basic demonstrations or single-connection experiments only. For production use, use a multi-threaded environment or run each client handler in a separate process.

Usage Examples

Example 1: Simple TCP Client

```
/* Connect to a web server and print the homepage */
sock = socketcreate()
rc = socketconnect(sock, "example.com", 80)
if rc = 0 then do
    call socketsend, sock, "GET / HTTP/1.0
Host: example.com

"
    response = ""
    do while socketisconnected(sock)
        chunk = socketrecv(sock, 1024)
        if chunk = "" then leave
        response = response || chunk
    end
    say response
end
else
    say "Connection failed:" socketlasterror(sock)
call socketclose, sock
```

Example 2: Simple TCP Server (Demonstrator Only!)

```
/* Echo server: Accept one connection, echo data, then exit */
sock = socketcreate()
rc = socketbind(sock, "0.0.0.0", 12345)
if rc = 0 then rc = socketlisten(sock, 1)
if rc = 0 then do
    say "Server listening on port 12345..."
    clientsock = socketaccept(sock)
    if clientsock > 0 then do
        say "Client connected:" socketpeerinfo(clientsock)
        do forever
            line = socketrecvline(clientsock)
            if line = "" then leave
            call socketsend, clientsock, line || "
"
        end
        call socketclose, clientsock
        say "Client disconnected."
    end
end
else
    say "Server setup failed:" socketlasterror(sock)
call socketclose, sock
```

> **Note:** In a real server, you would need to handle each accepted connection in a separate thread or process. This example only supports a single client connection at a time and will block until the client disconnects.

Example 3: Checking Errors

```
 sock = socketcreate()
rc = socketconnect(sock, "doesnotexist.invalid", 80)
if rc \= 0 then
    say "Error:" socketlasterror(sock)
call socketclose, sock
```